



Universiti Malaysia
KELANTAN

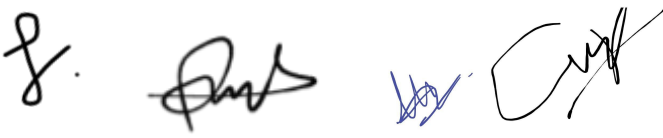
Course Code/Course Name	JIT21403 – Application Development		
Assessment:	Project – Part 2	Total Mark:	70
Date:	16/5/2024	Due Date:	24/6/2024
Name & Matric Number:	1) Norizzaty Binti Norsupardi (S22B0006) 2) Muhammad Amirul Ashraf Bin Mohd Nor (S22B0015) 3) Milashini a/p Saravanan (S22B0091) 4) Muhamad Arif Hilman Bin Zamzuri (S22B0016)		
Lecturer's Name:	Ts. Dr. Fakhitah Ridzuan		
I hereby declare that the work submitted for this assignment is our original work and that we have properly cited any sources of information used in the completion of this work. I understand that plagiarism, which includes copying and pasting information from online sources, submitting work completed by others, or any other form of cheating, is a serious academic offense and will not be tolerated.			
Signature: 			

Table of Contents

No.	Contents	Page
1.	Problem Statement	1
2.	Functional Requirement	3
3.	Use Case Diagram	5
4.	Code Explanation: OOP approach & Good Software Design Concept	6

1. Problem statement:

UMK students need help with organizing their bus schedules between Kampus Kota and Kemumin because of the inconsistent and ineffective bus timetable. Several problems are caused by the use of outdated communication methods and fragmented information in the current system. Students frequently fail to catch buses, endure long waiting periods, and express overall dissatisfaction with the bus service provided. The lack of an efficient and easily accessible method for sharing schedule changes worsens these difficulties. This leads to students feeling confused and frustrated because they rely on bus schedules that are timely and accurate to organize their daily activities.

Currently, certain schedule changes are notified via WhatsApp, however, this approach is unreliable and necessitates users to monitor modifications manually. Students frequently struggle to effectively plan their travels due to being uninformed about delays or changes in bus schedules caused by the absence of real-time updates. As a result, the current process is ineffective and untrustworthy, which leads students to have insufficient information to plan their timing.

Solution:

To tackle these inefficiencies, our suggestion is to create a mobile application for organizing bus timetables. This application's goal is to deliver a complete answer through the provision of instant notifications, customized timetables, and reminder notifications. The application will include a user-friendly interface to provide students with precise and current information, enhancing their travel experience.

Enhancements made to the mobile application:

- ☐ **Instant updates** on schedule changes, delays, and other crucial information will be provided by the app through real-time notifications. This function guarantees that students will be constantly informed about the current conditions of their bus routes, minimizing confusion and delays.
- ☐ **Customized Timetables** are important. Students can personalize their schedules to match their preferred travel times and specific routes. This method will help students better organize their travels.

- ☐ **Alerts/Notifications** will be sent by the app to remind users of their bus arrival time, lowering the chance of them missing it. This proactive function will improve timeliness and dependability for users.

- ☐ **Centralized Platform** has been used. Combining all schedule data into a single app removes the necessity for printed schedules and various communication methods. This platform will simplify the process, making it more convenient for students to obtain and understand bus schedule information.

In conclusion, the mobile application will completely change how UMK students handle their bus journeys from Kampus Kota to Kemumin. The app will improve students' commuting experience by fixing problems in the current system and offering a reliable, efficient, and user-friendly solution.

2. Functional Requirements

1. Homepage

1.1. Description: Central hub for accessing all features.

1.2. Search Button.

Function: Allows users to search for their desired destination.

Input: Destination name.

Output: Updated bus schedule for the specified destination.

2. Current Schedule

2.1. Description: Displays the current bus schedule.

2.2. Schedule Row.

Function: Shows the bus timings in a row format.

Input: None.

Output: List of bus timings with corresponding destinations and statuses (e.g., on time, delayed).

3. Full Schedule

3.1. Description: Provides the entire bus timetable.

3.2. View Full Schedule.

Function: Users can view the entire bus schedule.

Input: Click the full schedule location button.

Output: Display the complete bus timetable.

4. Set Reminder

4.1. Description: Allows users to manage their reminders.

4.2. Reminder Tile.

Function: Interface element for setting reminders.

Input: Click the tile.

Output: Navigate to the Add Reminder page.

4.3. Add Reminder

Function: Page to add new reminder notifications.

Input: Set the time and click 'Done'.

Output: Reminder notification before the bus arrives.

4.4. Remove Reminder

Function: Users can also remove set reminders.

Input: Click 'Remove' next to the reminder.

Output: Confirmation of reminder removal.

5. Notification

5.1. Description: Displays all notifications related to reminders.

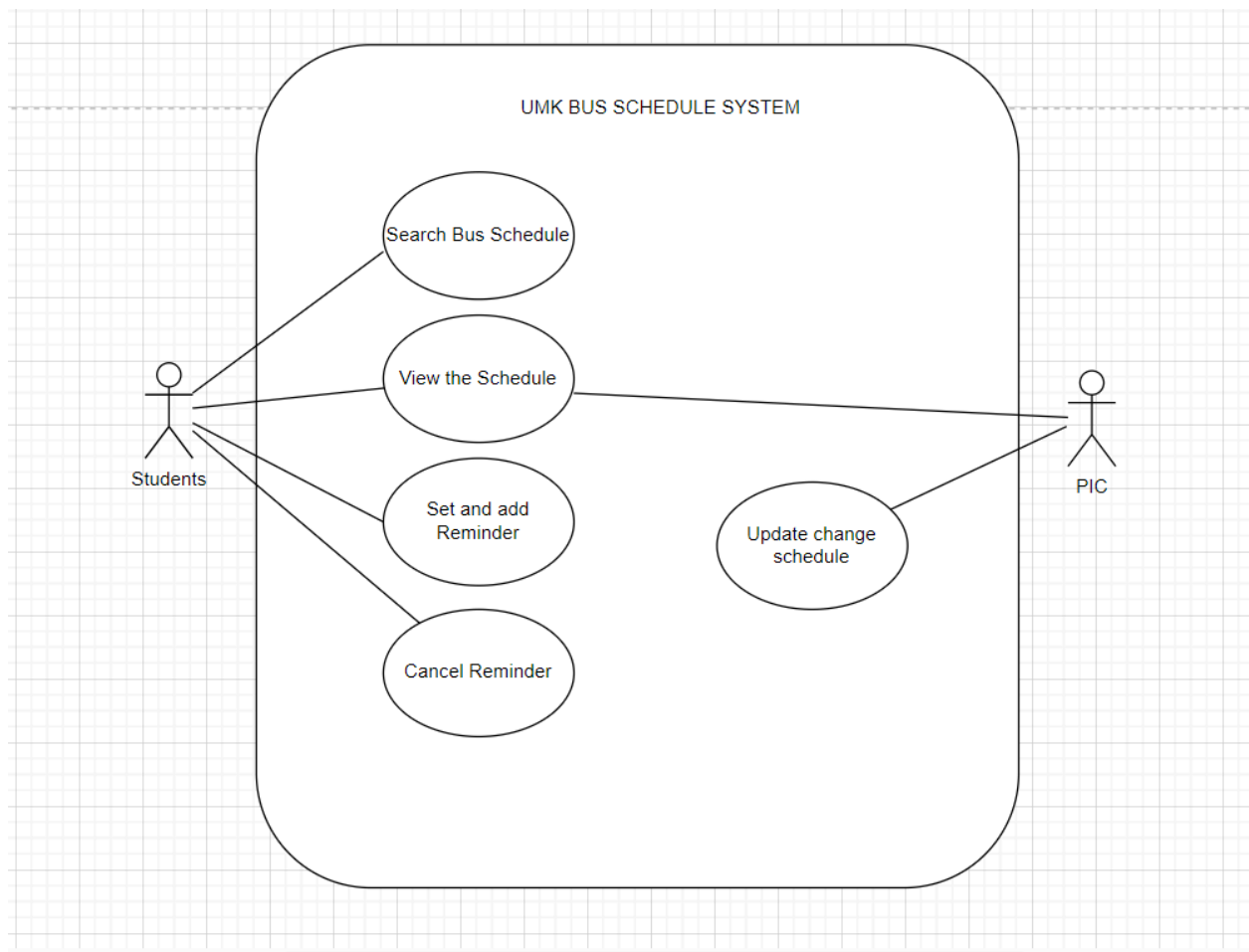
5.2. View Notifications.

Function: Users can view all recent notifications.

Input: None.

Output: List of notifications, including schedule changes, delays, and reminders.

3. Use Case Diagram



4. CODING EXPLANATION

4.1 Object Oriented approach involved

1. Encapsulation

```
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      debugShowCheckedModeBanner: false,  
      title: 'Bus Schedule App',  
      theme: ThemeData(  
        primarySwatch: Colors.orange,  
      ),  
      home: HomePage(),  
      routes: {  
        '/set_reminder': (context) => SetReminderPage(),  
        '/add_reminder': (context) => AddReminderScreen(),  
      },  
    );  
  }  
}
```

The above code shows encapsulation by use classes to arrange and organize the functionality of the program. Encapsulation, an essential concept of object-oriented programming, requires combining all of data and its associated methods within a class, thereby protecting the data from unauthorized access and improper use. The 'MyApp' class encapsulates the setup of the 'MaterialApp' widget, which includes the details of the title, theme, and routes. The 'HomePage' class, which is assumed to be stated elsewhere, includes the features and current state of the application's home page. The code includes additional imports ('Add_Reminder.dart', 'Set_Reminder.dart', and 'home_page.dart'), showing a modular approach where different features are enclosed within the related Dart programs. The implementation of modularity not only improves the overall structure of the codebase, but also simplifies the process of maintenance and comprehension. The code achieves encapsulation by encapsulating different components of the program in classes and separate files. This method ensures improved configuration, modularity, and protection of the internal state of each component.

2. Inheritance

```
class CustomAppBar extends StatelessWidget {
```

The CustomAppBar class displays inheritance as it extends the StatelessWidget class. In object-oriented programming, inheritance is another fundamental concept that enables a subclass to obtain attributes and methods from a superclass. In this context, CustomAppBar functions as the subclass, while StatelessWidget serves as the

superclass. The `CustomAppBar` class functions as a widget within the Flutter framework by inheriting the properties and methods of `StatelessWidget`. By inheriting from `CustomAppBar`, it is possible to use it as a `stateless widget`, which means it does not need to have permanent state and must modify the `construct` function in order to define its appearance. This function creates the user interface, which consists of an `AppBar` with a customized backdrop color, a logo image, and a title. By utilizing inheritance, the `CustomAppBar` is able to take advantage of the features offered by `StatelessWidget` while still specifying its own functionality and appearance.

3. Polymorphism

```
Expanded(
    child: DestinationDropdown(
        suggestions: suggestions,
        selectedDestination: selectedDestination,
        onDestinationSelected: _selectDestination,
    ),
),

NavigationTile(
    imagePath: "image/bust_stop_1.png",
    title: "Full Schedule",
    onTap: () {
        Navigator.push(
            context,
            MaterialPageRoute(
                builder: (context) => FullSchedulePage(),
            );
        },
    ),
),
);
```

Polymorphism in object-oriented programming enables the use of objects from various classes as if they were objects of a shared superclass. It allows for the use of one interface across different underlying frameworks (data types). In this instance, polymorphism enables the `HomePage` to interact with different widgets in normal ways, by calling their specific methods and parameters. For example, the classes `DestinationDropdown` and `NavigationTile` can be created and used within the same `ListView`, although having different purposes. The `onDestinationSelected` callback for `DestinationDropdown` and the `onTap` callback for `NavigationTile` show polymorphic activity. Both widgets utilize callback processes that activate when a

user interacts with them, while the purpose of these interactions different. The example of polymorphism in the Flutter framework is evident in its ability of interacting multiple widgets using an identical interface or method call, regardless of their specific implementations.

4.2 Good Software design concept

1. Separation of concern

The code for `SetReminderPage` shows the implementation of the idea of Separation of Concerns (SoC) by dividing the functionality into several components, each responsible for certain elements of the program. This enhances the code's modularity, simplifies maintenance, and improves readability.

- UI separation
 - App Bar

```
appBar: AppBar(  
  backgroundColor: Color.fromRGB0(241, 145, 86, 0.7),  
  title: Row(  
    mainAxisAlignment: MainAxisAlignment.center,  
    children: [  
      Image.asset(  
        "image/UMK_logo.png",  
        height: 60,  
      ), // Image.asset  
      SizedBox(width: 10),  
      Expanded(  
        child: Text(  
          "REMINDER",  
          style: TextStyle(fontSize: 22, fontWeight: FontWeight.bold),  
        ), // Text  
      ), // Expanded  
    ],  
  ),  
),
```

- Reminder List

```
Container(  
  padding: EdgeInsets.all(16),  
  decoration: BoxDecoration(  
    color: Colors.white,  
    borderRadius: BorderRadius.circular(10),  
    border: Border.all(  
      color: Colors.orange,  
      width: 2,  
    ),  
  ),  
,  
child: reminders.isEmpty  
  ? Text(  
    'No reminders set',  
    style: TextStyle(  
      color: Colors.grey,  
      fontStyle: FontStyle.italic,  
    ),  
  )  
  : Column(  
    children: reminders  
      .asMap()  
      .entries  
      .map((entry) => Column(  
        children: [  
          ReminderTile(  
            date: entry.value['date'] ?? '',  
            time: entry.value['time'] ?? '',  
            duration:  
              entry.value['duration'] ?? '',  
            onDelete: () =>  
              _removeReminder(entry.key),  
          ),  
          SizedBox(  
            height:  
              10), // Add spacing between reminders  
        ],  
      ),  
  ),  
),
```

- Logic Separation
 - Navigate and Data Handling

```
void _navigateAndDisplaySelection(BuildContext context) async {
  final result = await Navigator.pushNamed(context, '/add_reminder');

  if (result != null && result is Map<String, String>) {
    setState(() {
      bool alreadyExists = false;
      int index = 0;

      for (int i = 0; i < reminders.length; i++) {
        if (reminders[i]['date'] == result['date'] &&
            reminders[i]['time'] == result['time']) {
          alreadyExists = true;
          index = i;
          break;
        }
      }

      if (alreadyExists) {
        // Show dialog to confirm replacing the reminder
        showDialog(
          context: context,
          builder: (BuildContext context) {
            return AlertDialog(
              title: Text('Replace Reminder?'),
              content: Text(
                'This reminder already exists. Do you want to replace
it?'),
              actions: <Widget>[
                TextButton(
                  onPressed: () {
                    // Replace the reminder
                    setState(() {
                      reminders[index] = result;
                    });
                    Navigator.of(context).pop(); // Close the dialog
                  },
                  child: Text('Yes'),
                ),
                TextButton(
                  onPressed: () {
                    Navigator.of(context).pop(); // Close the dialog
                  },
                  child: Text('No'),
                ),
              ],
            );
          },
        );
      }
    });
  }
}
```

UI Component Separation involves splitting various parts of the user interface, such as the app bar and reminder list, into individual parts. This approach ensures that the responsibilities of each part are clearly stated. Regarding the concept of Logic Separation, the methods "_navigateAndDisplaySelection" handle separate logical activities, such as navigating to add a reminder and removing a reminder, respectively. By

keeping the functionality apart from the UI code, they are following the concept of Separation of Concerns (SoC).

By structuring the code in this manner, it fits to the idea of separation of concerns, resulting in a more modular, maintainable, and comprehensible program.

2. Single Responsibility Principle (SRP)

The Single Responsibility Principle (SRP) states that a class should have only one reason to change, pointing to that it should have only one job or responsibility. In the given code for the `NavigationTile` widget, SRP can be seen by the class being structured to encompass all the functionality associated with displaying a navigation tile.

```
Image.asset(  
  imagePath,  
  fit: BoxFit.cover,  
  width: double.infinity,  
), // Image.asset
```

(The class handles displaying an image using the `imagePath` provided.)

```
child: GestureDetector(  
  onTap: () => onTap(),
```

(The class uses a `GestureDetector` to manage tap interactions, calling the `onTap` callback when the tile is tapped)

By following to SRP, the `NavigationTile` class is focused on a single functionality (rendering a navigation tile with user interaction) and encapsulates all the necessary details to fulfill that responsibility. This makes the class easier to maintain and extend without disrupting other sections of the app. If there are modifications needed to how a navigation tile is presented or interacted with, they can be adjusted within this class without effecting other components.

3. Readability and Simplicity

Most of the provided code can be categorized as showing readability and simplicity. The code is structured in a way that makes each class and function easy to understand, maintain, and extend. Classes like ``CustomAppBar``, ``NavigationTile``, ``ReminderTile``, ``DestinationDropdown`` and more, are designed with clear, descriptive names and concise implementations, ensuring that their purpose and functionality are immediately apparent to anyone reading the code. These classes each focus on a single responsibility, encapsulating specific aspects of the UI or functionality, which helps to keep the codebase modular and organized.

This complying to the Single Responsibility Principle further improves the simplicity of the code, as each class handles only its designated task without unnecessary complexity. Moreover, the use of straightforward structures and well-chosen variable names contributes to the overall readability, making it easier for developers to quickly grasp the logic and make modifications when needed. By putting first readability and simplicity, the provided code sets a strong foundation for a maintainable and scalable software system, enabling easier collaboration and reducing the likelihood of errors.